Improving Multimodal Large Language Models Using Continual Learning

Anonymous ACL submission

Abstract

Generative large language models (LLMs) exhibit impressive capabilities, which can be further augmented by integrating a pre-trained vision model into the original LLM to create a multimodal LLM (MLLM). However, this integration often significantly decreases performance on natural language understanding and generation tasks, compared to the original LLM. This study investigates this issue using the LLaVA MLLM, treating the integration as a continual learning problem. We evaluate five continual learning methods to mitigate forgetting and identify a technique that enhances visual understanding while minimizing linguistic performance loss. Our approach reduces linguistic performance degradation by up to 15% over the LLaVA recipe, while maintaining high multimodal accuracy. We also demonstrate the robustness of our method through continual learning on a sequence of vision-language tasks, effectively preserving linguistic skills while acquiring new multimodal capabilities.

1 Introduction

Advances in integrating visual information with large language models (LLMs) have led to the development of multimodal large language models (MLLMs), excelling at many vision-language (VL) tasks (Wu et al., 2023; Yang et al., 2023; Li et al., 2023a; Alayrac et al., 2022; Chiang et al., 2023; Driess et al., 2023; Zhu et al., 2023a; Liu et al., 2023b; Luo et al., 2023a; Koh et al., 2023; Lin et al., 2023a; Liu et al., 2023a). Recent studies converge on a general recipe for developing MLLMs: Alignment of LLM token embeddings with visual embeddings followed by instruction-tuning on VL tasks like visual question answering (VQA) (Liu et al., 2023b). However, creating an MLLM often degrades the LLM's natural language understanding (NLU) and generation (NLG) performance, a phenomenon known as catastrophic forgetting (Li et al., 2023a; Driess et al., 2023). For instance, PaLM-E experienced an 87% drop in NLG performance over the base LLM (Driess et al., 2023). Similar forgetting has been noted for LLaVA (Lin et al., 2023a), but little work has addressed understanding and mitigating this issue.

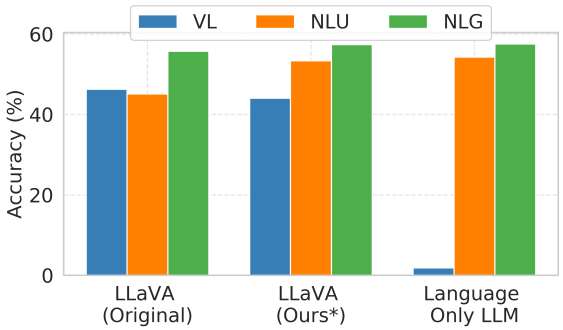


Figure 1: Summary results of our best method for training LLaVA1.5 compared to the unimodal base LLM, and an unmodified version of LLaVA1.5. All results are with Pythia 2.8B as the base LLM. Our approach results in nearly no loss in average vision-language (VL) performance, while providing a large increase in linguistic performance, on 1 NLG and 4 NLU tasks by absolute 8% and 2% respectively.

Here, we study mitigating the loss of linguistic abilities in the popular LLaVA MLLM using continual learning (CL) techniques designed to mitigate catastrophic forgetting (Parisi et al., 2019). In CL, a sequence of non-stationary tasks is learned, where we treat the first task as already learned by the base LLM followed by new VL tasks. We study these methods in two paradigms. In the first, we seek to recreate LLaVA 1.5 while mitigating linguistic forgetting through our methods, and the second task, where we sequentially learn each VL dataset in the LLaVA recipe.

This paper makes the following contributions:

1. Using the original LLavA 1.5 training recipe, we study linguistic forgetting in 7 MLLMs, including 5 built on the Pythia family of LLMs

to study the role of model scales and instruction tuning on such linguistic forgetting.

2. We study the effectiveness of 5 mitigation techniques for reducing linguistic forgetting, and show that our the best method improves accuracy for NLG, NLU, and VL tasks compared to the naive LLaVA recipe (see Fig. 1).
3. We pioneer studying continual learning for MLLMs by sequentially learning VL tasks, where we assess the efficacy of our continual learning techniques to mitigate catastrophic forgetting in this challenging scenario.

2 Background

2.1 Multimodal Large Language Models

Open source MLLMs integrate pre-trained vision encoders into LLMs (see Bordes et al. (2024) for review). Notable works include PaLM-E (Driess et al., 2023), LLaVA 1.5 (Liu et al., 2023b,a), MiniGPT4 (Zhu et al., 2023a), LaVIN (Luo et al., 2023a), Visual ChatGPT (Wu et al., 2023), BLIP-2 (Li et al., 2023a), Qwen-VL (Bai et al., 2023), Flamingo (Alayrac et al., 2022), FROMAGe (Koh et al., 2023), and VILA (Lin et al., 2024). PaLM-E is an embodied MLLM that is capable of grounded reasoning for VL and language tasks. Its authors reported that smaller versions of the model had severe catastrophic forgetting on NLG/NLU tasks compared to its base LLM, except for their 562B parameter model. MiniGPT-4 uses a trainable fusion layer to link the LLM to a vision encoder, with both the LLM and vision encoder frozen. Similar to MiniGPT-4, Qwen-VL and Qwen-VL-Chat models use a one-layer cross-attention module to inject vision embeddings into an LLM. BLIP-2 uses a Q-Former to map visual information into the input space of LLM. LaVIN introduces Mixture-of-Modality Adaptation to extend the multi-modal capabilities of LLMs with a vision encoder. FROMAGe demonstrated good performance on image captioning and image-text retrieval tasks using trainable input and output linear layers, while keeping the vision encoder and LLM frozen. In this paper, we study LLaVA, which shares many similarities with these models and is described in detail in Sec. 3.1.

While many MLLMs have been proposed, most papers focus only on VL performance and do not measure whether this resulted in any loss of NLU/NLG performance, including the original MiniGPT-4 and LLaVA papers. Notable exceptions are PaLM-E, LaVIN, and FROMAGe. However, they do not study methods for mitigating forgetting when an LLM is converted into an MLLM. Given the tremendous drop in NLU/NLG performance in the PaLM-E paper for smaller LLMs, which are the most important for consumer applications, addressing this problem could significantly improve the utility of MLLMs. The only work that has attempted to address NLU/NLG forgetting is VILA (Lin et al., 2024), which did so by including MMC4 in their training recipe, which is a dataset that interleaves text and images (Zhu et al., 2023b). To the best of our knowledge, there is no work studying how to mitigate linguistic forgetting in MLLMs using methods from CL, which is what we address in this work.

2.2 Continual Learning

Creating an MLLM maps directly onto a CL scenario involving learning two tasks sequentially, where the first task is acquiring NLU/NLG capabilities and the second is acquiring VL capabilities. Although some recent works have studied CL with unimodal LLMs (Luo et al., 2023b; Lin et al., 2023b), our work is the first to treat creating an MLLM as a CL problem. While there are many CL techniques, two general families of methods are most appropriate for MLLMs: rehearsal and regularization. Rehearsal methods store or reconstruct previously observed data that is mixed into new data to overcome catastrophic forgetting (Hayes et al., 2021, 2020; Chaudhry et al., 2019; Harun et al., 2023; Hou et al., 2019; Rebuffi et al., 2017). The primary challenge with using it for MLLMs is that the training sets for many open-source LLMs have not been released, and even if they were released, determining what to rehearse may be intractable due to the large size of these datasets. Regularization methods adjust training processes and impose constraints on weight updates to reduce forgetting (Chaudhry et al., 2018b,a; Dhar et al., 2019; Kirkpatrick et al., 2017; Zenke et al., 2017). We focus on lightweight regularization methods in this work that can scale to MLLMs.

2.3 Parameter Efficient Tuning (PET)

Parameter Efficient Tuning (PET) methods are an efficient alternative to full fine-tuning (Houlsby et al., 2019). They update fewer parameters, thereby reducing computational costs while maintaining performance. In Harun and Kanan (2023), LoRA was shown to effectively retain performance

in ImageNet pre-trained models as new classes were continually learned. Inspired by this, we study the effectiveness of LoRA (Hu et al., 2021) in mitigating catastrophic forgetting while learning VL tasks.

3 Methods

3.1 The LLaVA MLLM

We study LLaVA 1.5, henceforth referred to as LLaVA, which is one of the most widely used multimodal training protocols. LLaVA has the following components:

- **Visual Encoder:** Following earlier implementations (Liu et al., 2024, 2023a), we use a pre-trained ViT-L/14 from CLIP which takes an image resolution of 336px as the vision encoder, which is kept frozen throughout the training to ensure stability and prevent overfitting to the initial training tasks.
- **LLM:** We study 9 LLMs of various sizes and training protocols (see Sec. 3.1.1).
- **Alignment Network:** To inject other modalities into LLaVA, it uses a linear alignment network that projects embeddings from vision into the representational space of the text tokens (Merullo et al., 2022). This network has two layers that map each vision embedding to the text embedding space.

We follow the standard LLaVA 1.5 training recipe, with details, including optimizer settings and hardware, provided in Appendices B and C.2.

3.1.1 LLMs Studied

We study 9 choices for LLaVA's LLM: 6 Pythia models (Biderman et al., 2023), Phi2 (3B) (Gunasekar et al., 2023), and 2 LLaMA 2 (7B) models. The 6 Pythia models are of various scales (160M, 410M, 1B, 1.4B, and 2.8B parameters), where there are 2 versions of Pythia (1.4B), one of which is instruction fine-tuned. Likewise, we study the original version of LLaMA 2 and an instruction fine-tuned version, Vicuna-1.5 7B (Chiang et al., 2023), which was used in LLaVA 1.5.

Phi2 (Li et al., 2023b) has been trained on the same dataset as Phi1 (Gunasekar et al., 2023), which includes a curated selection of "textbook quality" data from the web (6 billion tokens) and an additional 1 billion tokens of synthetically generated textbooks and exercises created using GPT-3.5 (Brown et al., 2020). The Phi series are among the most performative models in the under 7B parameter size class (Li et al., 2023b). Pythia (Biderman et al., 2023) comprises two sets of 8 models, each corresponding to two datasets. For every model size, one set is trained on the Pile dataset (Gao et al., 2020), while the other set is trained on a version of the Pile where global deduplication has been applied. The granular model scaling suite of Pythia is particularly useful for our study. We select the de-duplicated set of Pythia models. LLaMA 2 is reportedly trained on a mix of publicly available online data, but specific details are not available (Touvron et al., 2023). LLaVA 1.5 reports its best performance with instruction fined-tuned LLaMA 2 LLMs (Liu et al., 2023a).

3.2 Continual Learning Methods

To mitigate catastrophic forgetting in MLLMs, we examine several methods:

1. **Naive Fine-Tuning** corresponds to the original LLaVA method with no modifications.

2. **LoRA** keeps the original LLM weights frozen and learns low-rank updates for them (Hu et al., 2021). Following LoRA's recommended protocol, we inject LoRA weights into all LLM linear layers. After each task is learned, the LoRA weights are merged into the LLM. Details are given in Appendix C.4.2.

3. **Soft Targets** was proposed in Harun and Kanan (2023) to reduce forgetting in CL. Rather than using hard targets for training, dynamic label smoothing is used to reduce the severity of the training loss under distribution shift in CL. Let Y be the hard target vector and $f(X)$ be the model's logits for input X. The soft target vector $\hat{Y}$ is given by $\hat{Y} = \alpha \cdot \text{softmax}(f(X)) + (1 - \alpha)Y$, where α controls the interpolation. We discuss the choice of α in Appendix C.4.3.

4. **mSGM** is based on SGM, which combines soft targets, weight initialization, and LoRA to mitigate catastrophic forgetting (Harun and Kanan, 2023). We use a version that omits the weight initialization and output layer freezing process used by SGM for class incremental learning since the output vocabulary is static and output layers are used for the causal generation of LLMs.

5. **Rehearsal (Experience Replay)** is an effective method for CL that involves storing data

from earlier tasks and mixing it with data from new tasks. We study it in our CL experiments since we do not have any stored data for task 1 (training the base LLM). We study storing 1% and 10% of randomly selected samples from each previous task, excluding task 1.

In all methods, the alignment layer and LLM are trained and the ViT is frozen.

3.3 Evaluation Datasets

To facilitate analysis across tasks, all of our evaluation sets use the same metric: accuracy.

Natural Language Evaluation: We use six datasets. For NLG, we use Lambada (Paperno et al., 2016), which is the only NLG dataset that uses accuracy for evaluation. For NLU, we use ARC-Easy (Clark et al., 2018), ARC-Challenge (Clark et al., 2018), Winogrande (Sakaguchi et al., 2021), and WSC (Sakaguchi et al., 2021) for NLU.

Vision-Language Evaluation: We use the test sets corresponding to each VL dataset used for training LLaVA: VQAv2 and GQA for general VQA tasks (Goyal et al., 2017; Hudson and Manning, 2019), TextVQA OCR (and Pure) for OCR tasks (Singh et al., 2019), and RefCOCO for referential expression generation tasks (Kazemzadeh et al., 2014). We use the *slim* version of all test datasets to make our experiments tractable [1]. Additional details are given in Appendix D.

3.4 Measuring Performance

To assess performance, we compute the forgetting metric Δ, where a positive value indicates forgetting and a negative value indicates learning the next task enhances the performance of previously acquired tasks (or a backward transfer (Zenke et al., 2017)). The performance change, Δ, on task t after training on task k is defined as:

$$\Delta_t(k) = \omega_t(1) - \omega_t(k), \quad \forall t < T \qquad (1)$$

where T is the total number of tasks, and ω_t is the harmonic mean of the accuracy values of the evaluation datasets for task t. We use the harmonic mean because it is dominated by the accuracy of the *worst-performing* dataset for the task, thereby emphasizing the need to perform well and avoid forgetting for all of the datasets.

[1] https://github.com/TRI-ML/vlm-evaluation

4 Experiments

In our experiments, we treat training a MLLM as a CL problem, where the system learns a sequence of tasks from 1 to T. Task 1 always consists of training the LLM, where we assume the LLM has already been trained and we do not know the provenance of the training data or the exact methods to create it, which is true of many commonly used LLMs (e.g., Llama 2 and Llama 3). In our experiments on analyzing and mitigating linguistic forgetting, there are 2 tasks: 1) learning the base LLM, and 2) learning the mixture of VL datasets. In CL experiments, there are 5 tasks where the first is training the base LLM, and then each VL task is sequentially learned.

Following the standard LLaVA recipe, for all experiments, task 2 begins by training the VL alignment network using the LLaVA-595 CC-SBU captioning dataset. The network is trained to generate captions with an auto-regressive loss, with the vision encoder and LLM kept frozen. Because only the alignment network is trained, the model has a modicum of visual ability without impairing its language capabilities. Subsequently, the LLM and alignment layer are trained for tasks 2 to T. For Secs. 4.1 and 4.2, their two-task setup is identical to LLaVA 1.5's training protocol. Additional implementation details are given in Appendix C.

4.1 Analyzing Linguistic Forgetting

Linguistic forgetting has not been studied in LLaVA, so before assessing CL methods, we first measure linguistic forgetting and VL performance for all of our LLMs using the standard LLaVA training recipe; which uses naive fine-tuning. Using this recipe, our 9 LLMs are transformed into MLLMs.

Overall Results. Our overall results are given in Fig. 2. Five models suffer from linguistic forgetting, while surprisingly, four have increased NLU/NLG accuracy due to positive transfer from the VL tasks. We next dive into these results, where our analysis of NLU/NLG forgetting as a function of model scale is provided in a later subsection.

Positive Backwards Transfer & Analysis of NLU vs. NLG. Datasets. In Table 1 and Fig. 5, we study the impact of forgetting on NLU vs. NLG tasks. We find that forgetting is much more severe for our one NLG task than the NLU tasks. All MLLMs exhibit greater NLG forgetting than NLU forgetting as shown in Table 1. In terms of

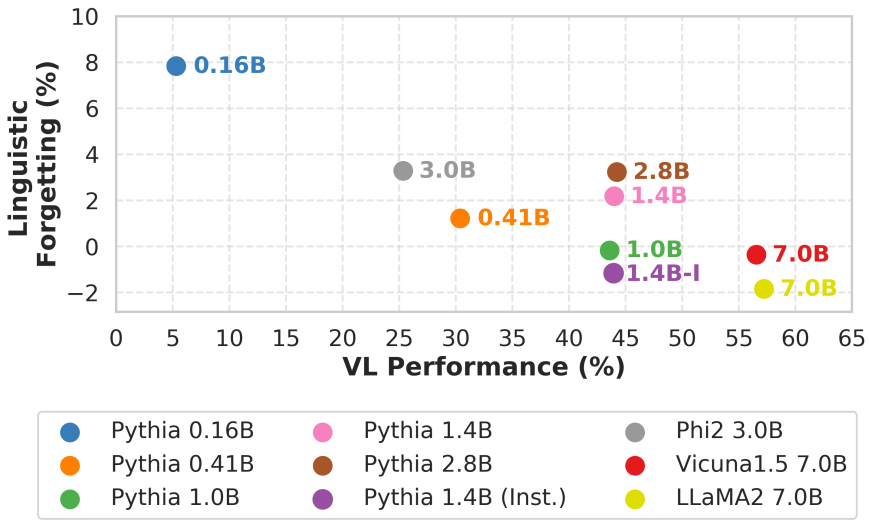


Figure 2: Linguistic forgetting vs. VL performance for 9 MLLMs trained with the LLaVA recipe. Five models exhibited linguistic forgetting, and four had negative linguistic forgetting values, indicating that VL training resulted in positive transfer to language tasks.

model size, typically smaller models show higher NLG forgetting, which is consistent with PaLM-E's observations. Across MLLMs, NLG forgetting is much more severe compared to NLU, and the NLU datasets are the source of the cases of positive backward transfer (negative Δ). We posit this may be due to the additional common-sense reasoning and world knowledge encoded in visual-language tasks and instructions, which is relevant for the NLU tasks. We include additional results on the NLU and NLG forgetting in Appendix E.1.

Table 1: **NLU vs NLG Forgetting:** We look at the composition of forgetting between NLU, and NLG tasks for models after LLaVA training. Negative Δ indicates positive backward-transfer which is desirable. The letter, I denotes the instruction-tuned model.

LLM	Scale (B)	NLU Δ ↓	NLG Δ ↓
Pythia	0.16	0.94	12.01
	0.41	–1.19	8.62
	1.0	–1.63	4.95
	1.4	0.55	8.07
	2.8	1.74	9.18
	1.4 (I)	–1.20	–1.01
Phi2	3.0	2.60	4.39
Vicuna1.5	7.0	–0.98	2.04
LLaMA2	7.0	–2.15	–0.43
Average	—	–0.15	5.31

Table 2: Results for LLaMA2 and Pythia after LLaVA training, with and without instruction fine-tuning. Negative Δ indicates positive backward transfer.

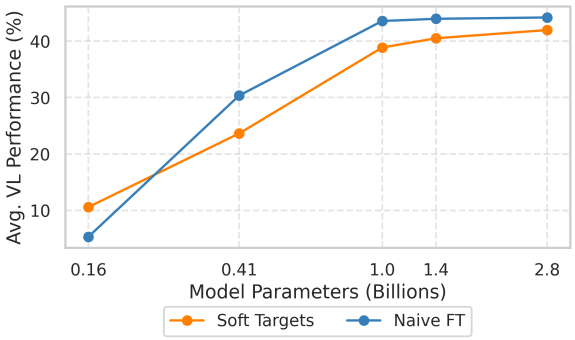
Base LLM	Instr.	VL Avg. Acc ↑	NL Avg. Acc ↑	NL Avg. Δ ↓
LLaMA2 (7B)	✗	**57.2**	**66.2**	**–1.8**
	✓	56.6	64.4	–0.4
Pythia (1.4B)	✗	**44.0**	**45.5**	2.2
	✓	43.9	41.4	**–1.2**

The Impact of Instruction Fine-Tuning. Previous work on continual learning in large language models (LLMs) has shown that instruction fine-tuning mitigates catastrophic forgetting (Luo et al., 2023b). This study extends the investigation to multimodal language models (MLLMs), analyzing LLaVA models trained with Pythia (1.4B) and LLaMA2 (7B), and their instruction fine-tuned versions: Vicuna1.5, used in the original LLaVA paper (Liu et al., 2023a), and an instruction-tuned Pythia (1.4B)[2]. As shown in Table **??**, non-instruction-tuned LLMs exhibit minor increases in visual-linguistic (VL) accuracy ($< 1\%$) and a larger difference in natural language understanding/generation (NLU/NLG) accuracy compared to their instruction-tuned counterparts. Instruction-tuned LLaMA2 (7B) and Pythia (1.4B) models show minimal or even negative linguistic forget-

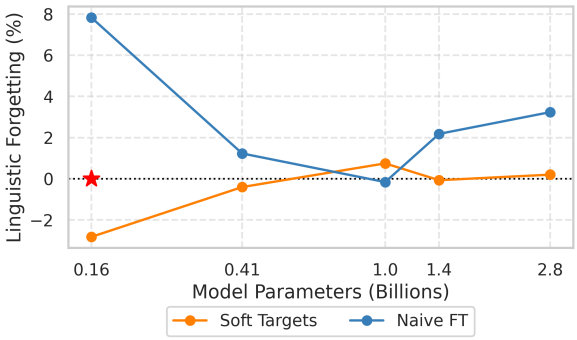
[2] https://huggingface.co/lambdalabs/pythia-1.4b-deduped-synthetic-instruct

Table 3: Results for alternatives to naive fine-tuning for transforming Pythia 160M into an MLLM.

Model	Vision-Language (VL) ↑				VL Avg.	NL Avg.	
	VQAv2	TextVQA OCR	TextVQA Pure	GQA	Acc ↑	Δ ↓	Acc ↑
Pythia (160M)	0.00	0.00	0.00	0.00	0.00	–	**32.61**
Naive FT	30.32	2.40	3.83	22.17	5.29	7.83	24.78
LoRA	28.97	1.02	1.74	17.97	2.42	**1.69**	30.92
mSGM	28.39	1.37	2.71	17.48	3.36	2.68	29.93
Soft Targets	**32.67**	**6.92**	**6.10**	**25.39**	**10.57**	2.83	29.78



(a) Avg. VL performance with varied model size



(b) Linguistic forgetting with varied model size

Figure 3: **Vision-language and linguistic forgetting for varying model sizes**. We evaluate the average VL performance and linguistic forgetting for Pythia models in the 160M to 2.8B scale, after training on LLaVA. Negative forgetting refers to a positive backward transfer in NL performance after multimodal training.

ting, indicating positive backward transfer. Conversely, base LLMs display greater variability, with base Pythia experiencing more forgetting than base LLaMA2, possibly due to differences in their pre-training datasets affecting post-multimodal training linguistic performance. Instruction tuning significantly reduces NLG forgetting in Pythia (1.4B) and induces positive backward transfer from VL to NLG, with a similar trend observed in NLU tasks. This suggests that instruction-tuned LLMs exhibit little to no forgetting, likely due to the distributional similarity between text-only instruction tuning tasks and visual-instruction tuning, helping to mitigate catastrophic forgetting during multimodal training.

4.2 Mitigating Linguistic Forgetting

Following the results in Sec. 4.1, we study the efficacy of our mitigation methods toward reducing linguistic forgetting. Due to limited computational resources, we only exhaustively tested the mitigation methods with Pythia (160M), which was selected due to its small size. We then evaluate the best method using all Pythia models in Sec. 4.2.

Our results with various mitigation methods on LLaVA training of Pythia (160M) are given in Table 3. Soft Targets has the highest accuracy across VL datasets with the least linguistic forgetting. LoRA and mSGM better preserve NLU/NLG performance but at the cost of decreased VL accuracy compared to naive fine-tuning.

Analyzing the Role of Parameter Count. In Driess et al. (2023), larger models had less catastrophic forgetting than smaller ones for NLG/NLU, where their 12B parameter model lost 87.3%. In comparison, their 562B parameter model only lost 3.9%. The opposite result was found in Luo et al. (2023b), where smaller models exhibited *less* forgetting than larger ones.

We analyze this phenomenon in the Pythia family of models, where we evaluate naive fine-tuning and Soft Targets, which was the best method for Pythia (160M). Results are given in Fig. 3. Across scales, Soft Targets has zero or negative linguistic forgetting over the naive fine-tuning method used in the original LLaVA paper, with competitive VL performance compared to naive fine-tuning. Soft Targets achieved positive backward transfer in the low model size regime (0.16-0.41B) and no forgetting for larger sizes (>0.41B). In contrast, naive fine-tuning had severe forgetting in the 0.16-0.41B regime and reduced forgetting when the model size

Table 4: **Continually learning LLaVA Tasks with Pythia 410M:** We report task-wise accuracy and forgetting of each mitigation method across VL and NLU/NLG tasks using harmonic mean.

Model	Task 2 (Instruct)		Task 3 (VQA)		Task 4 (OCR)		Task 5 (Ref)	
	VL (A ↑)	NL (Δ ↓)	VL (A ↑)	NL (Δ ↓)	VL (A ↑)	NL (Δ ↓)	VL (A ↑)	NL (Δ ↓)
Naive-FT	0.37	**0.58**	**44.22**	12.21	**16.67**	4.95	0.48	7.70
Soft Targets (ST)	**0.45**	0.81	0.16	14.67	10.23	5.41	0.31	10.90
LoRA	0.20	1.38	37.46	1.23	14.03	2.50	9.59	4.36
mSGM	0.10	1.11	36.31	1.57	11.69	**1.40**	0.32	6.32
Rehearsal (1%)	0.37	**0.58**	37.74	10.90	3.47	7.41	3.55	7.65
mSGM + Reh. (1%)	0.10	1.11	35.28	**0.73**	12.38	2.25	**10.21**	**2.77**

exceeded 1B parameters.

4.3 Continually Learning VL Tasks

Table 5: **Continual LLaVA Setup**: The LLaVA1.5 data mixture is split into groups of vision-language (VL) tasks. VQA (OE & OK) refers to open-ended and outside-knowledge VQA tasks.

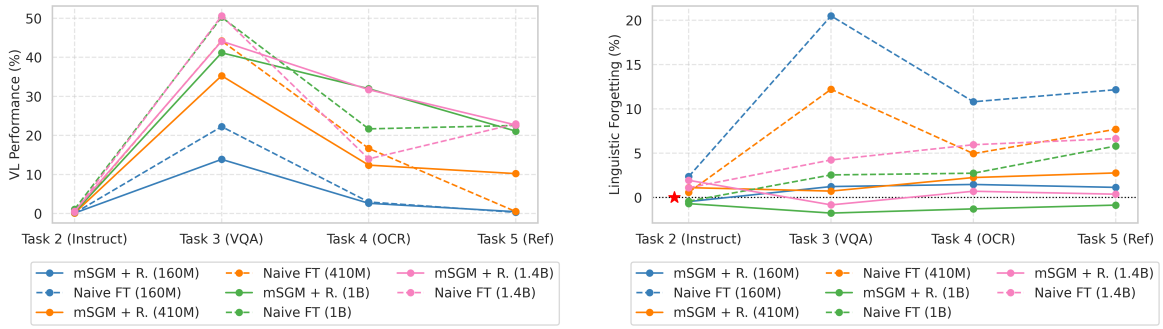
Task Type	Task	Data	Size
Pre-Training	1	LLM Pre-Training*	–
Instruct Tuning	2	CC-LAION-SBU	558K
		LLaVA-Inst, ShareGPT	198K
VQA (OE & OK)	3	VQA2	83K
		OKVQA	9K
		A-OKVQA	66K
		GQA	72K
VQA (OCR)	4	OCRVQA	80K
		TextCaps	22K
Referential Grounding	5	RefCOCO	48K
		VisualGenome	86K

We next turn to continually learning each of the VL datasets used to train LLaVA 1.5, where we group the datasets based on the type, domain of images, and questions. The task order is provided in Table 5, with additional details provided in Appendix D. Given our limited computational budget, we exhaustively studied our CL methods only for the smaller-scale Pythia 410M LLM, and then we evaluated the best-performing mitigation method for all of the Pythia LLMs (from 160M to 1.4B parameters).

Results for Pythia 410 are given in Table 4. In terms of NLU/NLG forgetting, all mitigation methods showed efficacy in reducing forgetting across the sequence of tasks. Overall, mSGM with and without rehearsal achieves the least linguistic forgetting while maintaining the highest VL performance across all baselines. Regarding VL performance, naive fine-tuning achieves better performance at the cost of high linguistic forgetting. In contrast, mSGM achieves competitive VL performance and sometimes even surpasses it, e.g., for RefCOCO, an especially challenging VL task. Model scaling results are given in Fig. 4.

4.3.1 Observations On Catastrophic Forgetting in Continual Training

We now analyze how the mitigation methods scale with larger model parameter counts, towards mitigating linguistic forgetting while maximizing VL performance during the continual LLaVA training. We compare mSGM + Rehearsal (our best CL method on the continual LLaVA training setting per Table 4) to the Naive LLaVA approach (Naive FT). We observe that naive fine-tuning leads to a sharp and consistent drop in NL performance as continual multi-modal training proceeds (Fig. 4b). Different model sizes result in varying degrees of catastrophic forgetting. On the contrary, our best mitigation method, mSGM with rehearsal, achieves little to no forgetting across all tasks and multiple model scales. Remarkably, while Naive FT (1B) suffers from forgetting, mSGM + Rehearsal (1B) achieves positive backward transfer across all tasks, which is notable in continual training setting. As shown in the earlier section (sec. 4.2), mSGM mitigation mechanisms especially Soft Targets showed consistent efficacy in mitigating forgetting. The Naive FT LLaVA training at the 160M model size exhibits significantly large forgetting as opposed to mSGM. Additionally, Naive FT appears to be sensitive to tasks as it intensifies forgetting for some tasks whereas mSGM maintains minimal forgetting across all tasks. Notably, while preventing linguistic forgetting mSGM rivals Naive FT in VL performance as shown in Fig. 4a. Overall, we see that the mSGM + Rehearsal method achieves little to no forgetting with near-maximal VL performances across multiple model scales.



(a) Continual VL Performance with varied model size

(b) Continual Linguistic Forgetting with varied model size

Figure 4: **Continual Learning LLaVA Tasks: Vision-language performance and Linguistic forgetting for varying model sizes**. We evaluate mSGM + Rehearsal (1%) and LLaVA Naive-FT on the Continual Learning setup, with varying base LLMs: Pythia models from 160M to 1.4B scale. Task 2 'Instruct', does not include any academic VL datasets used for evaluation, and therefore evaluated VL performance is close to 0 for Task 2 in Figure 4a.

5 Discussion

This work presents one of the first studies of linguistic forgetting in multimodal large language models (MLLMs), particularly for model scales in the small-medium range (<7B parameters). We study a number of continual/mitigation methods that we show through experiments mitigate the linguistic forgetting effect significantly compared to the original LLaVA recipe while having competitive VL performance. We analyze and show that this benefit holds across model scales and even in the continual LLaVA training setting. Multimodal abilities are essential for many applications, and our findings show that naive multimodal fine-tuning significantly degrades prior linguistic abilities. Our mitigation method successfully preserves linguistic abilities while achieving competitive VL performance. By laying the foundation for continual MLLM study, we aim to inspire future research to develop more robust and capable mitigation approaches.

6 Limitations

In this work, we rigorously studied catastrophic forgetting in various multimodal LLMs and showed that MLLMs do forget current and previous knowledge including NLG and NLU in CL settings. We evaluated different prevention mechanisms to improve the performance of MLLMs. While there are limitations in our exploration, we believe our study will pave the way towards building robust MLLMs that accumulate multi-modal knowledge without forgetting.

We studied LLaVA, but we assume our findings would apply to similar models (Liu et al., 2024, 2023a; Lin et al., 2023a). We studied the most popular PET methods such as LoRA, however, there are other methods (Han et al., 2024) that we did not study. Future work could study other methods and investigate their ability to reduce forgetting in MLLMs.

We experimented with 9 MLLMs and our experiments were computationally demanding. We studied CL with 5 tasks. While more versatile and large-scale CL settings with more tasks and task orderings are ideal for investigating the generality of our method, we had to limit our exploration due to limited resources. Future work might investigate how learning more tasks affects forgetting in MLLMs.

We only studied vision, but some MLLMs have additional modalities (Driess et al., 2023) and it is not clear if our methods will be as effective for those. Our current work mainly focuses on MLLMs with multi-modal inputs and text-only outputs. It would be interesting to study MLLMs with both multi-modal inputs and outputs for multimodal dialogue in CL settings. FROMAGe (Koh et al., 2023) studied this in a non-CL setting with frozen LLM, exploration of different CL methods is an exciting future direction to build a multi-modal chatbot or agent that can continually and seamlessly interact with users.

LLaVA is the most widely adopted MLLM framework within the research community. At present, there are no continual MLLM frameworks that align with the specific requirements of our study. Therefore, to ensure a consistent and fair comparison among various MLLMs, we have cho-

sen to utilize LLaVA. While this decision may result in under-training MLLMs for certain VL tasks, it ensures that the comparative analysis remains equitable and unbiased across all methods.

Due to the limited compute budget, we could not study super large models with parameters more than 7B, it would be interesting to examine how model size impacts catastrophic forgetting in larger LLMs. Note that PaLM-E found that their enormous model had minimal forgetting. For instance, PaLM-E (12B) exhibits 87.3% forgetting whereas the largest one, PaLM-E (562B) has only 3.9% forgetting. It is unclear how beneficial our methods would be in that scenario.

References

Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. 2022. Flamingo: a visual language model for few-shot learning. *Advances in Neural Information Processing Systems*, 35:23716–23736.

Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond.

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. 2023. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR.

Florian Bordes, Richard Yuanzhe Pang, Anurag Ajay, Alexander C Li, Adrien Bardes, Suzanne Petryk, Oscar Mañas, Zhiqiu Lin, Anas Mahmoud, Bargav Jayaraman, et al. 2024. An introduction to vision-language modeling. *arXiv preprint arXiv:2405.17247*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Arslan Chaudhry, Puneet K Dokania, Thalaiyasingam Ajanthan, and Philip HS Torr. 2018a. Riemannian walk for incremental learning: Understanding forgetting and intransigence. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 532–547.

Arslan Chaudhry, Marc'Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. 2018b. Efficient lifelong learning with a-gem. *arXiv preprint arXiv:1812.00420*.

Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K Dokania, Philip HS Torr, and Marc'Aurelio Ranzato. 2019. Continual learning with tiny episodic memories. *arXiv preprint arXiv:1902.10486, 2019*.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

Prithviraj Dhar, Rajat Vikram Singh, Kuan-Chuan Peng, Ziyan Wu, and Rama Chellappa. 2019. Learning without memorizing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5138–5146.

Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. 2023. Palm-e: An embodied multimodal language model. *ICML*.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A framework for few-shot language model evaluation.

Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. 2017. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6904–6913.

Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. 2023. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*.

Zeyu Han, Chao Gao, Jinyang Liu, Sai Qian Zhang, et al. 2024. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*.

Md Yousuf Harun, Jhair Gallardo, Tyler L. Hayes, Ronald Kemker, and Christopher Kanan. 2023. SIESTA: Efficient online continual learning with sleep. *Transactions on Machine Learning Research*.

Md Yousuf Harun and Christopher Kanan. 2023. Overcoming the stability gap in continual learning. *arXiv preprint arXiv:2306.01904*.

Tyler L Hayes, Kushal Kafle, Robik Shrestha, Manoj Acharya, and Christopher Kanan. 2020. Remind your neural network to prevent catastrophic forgetting. In *European Conference on Computer Vision*, pages 466–483. Springer.

Tyler L Hayes, Giri P Krishnan, Maxim Bazhenov, Hava T Siegelmann, Terrence J Sejnowski, and Christopher Kanan. 2021. Replay in deep learning: Current approaches and missing biological elements. *Neural Computation*, 33(11):2908–2950.

Saihui Hou, Xinyu Pan, Chen Change Loy, Zilei Wang, and Dahua Lin. 2019. Learning a unified classifier incrementally via rebalancing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 831–839.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799. PMLR.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Drew A Hudson and Christopher D Manning. 2019. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6700–6709.

Damjan Kalajdzievski. 2023. A rank stabilization scaling factor for fine-tuning with lora. *arXiv preprint arXiv:2312.03732*.

Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna, Percy Liang, Thomas Kollar, and Dorsa Sadigh. 2024. Prismatic vlms: Investigating the design space of visually-conditioned language models. *arXiv preprint arXiv:2402.07865*.

Sahar Kazemzadeh, Vicente Ordonez, Mark Matten, and Tamara Berg. 2014. Referitgame: Referring to objects in photographs of natural scenes. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 787–798.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526.

Jing Yu Koh, Ruslan Salakhutdinov, and Daniel Fried. 2023. Grounding language models to images for multimodal inputs and outputs. *ICML*.

Ranjay Krishna, Yuke Zhu, Oliver Groth, Justin Johnson, Kenji Hata, Joshua Kravitz, Stephanie Chen, Yannis Kalantidis, Li-Jia Li, David A Shamma, et al. 2017. Visual genome: Connecting language and vision using crowdsourced dense image annotations. *International journal of computer vision*, 123:32–73.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023a. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. *arXiv preprint arXiv:2301.12597*.

Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023b. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*.

Ji Lin, Hongxu Yin, Wei Ping, Yao Lu, Pavlo Molchanov, Andrew Tao, Huizi Mao, Jan Kautz, Mohammad Shoeybi, and Song Han. 2023a. Vila: On pre-training for visual language models. *arXiv preprint arXiv:2312.07533*.

Ji Lin, Hongxu Yin, Wei Ping, Pavlo Molchanov, Mohammad Shoeybi, and Song Han. 2024. Vila: On pre-training for visual language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26689–26699.

Yong Lin, Lu Tan, Hangyu Lin, Zeming Zheng, Renjie Pi, Jipeng Zhang, Shizhe Diao, Haoxiang Wang, Han Zhao, Yuan Yao, et al. 2023b. Speciality vs generality: An empirical study on catastrophic forgetting in fine-tuning foundation models. *arXiv preprint arXiv:2309.06256*.

Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. 2023a. Improved baselines with visual instruction tuning. *arXiv preprint arXiv:2310.03744*.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023b. Visual instruction tuning. In *NeurIPS*.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2024. Visual instruction tuning. *Advances in neural information processing systems*, 36.

Gen Luo, Yiyi Zhou, Tianhe Ren, Shengxin Chen, Xiaoshuai Sun, and Rongrong Ji. 2023a. Cheap and quick: Efficient vision-language instruction tuning for large language models. *arXiv preprint arXiv:2305.15023*.

Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. 2023b. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *arXiv preprint arXiv:2308.08747*.

Kenneth Marino, Mohammad Rastegari, Ali Farhadi, and Roozbeh Mottaghi. 2019. Ok-vqa: A visual question answering benchmark requiring external knowledge. In *Proceedings of the IEEE/cvf conference on computer vision and pattern recognition*, pages 3195–3204.

Jack Merullo, Louis Castricato, Carsten Eickhoff, and Ellie Pavlick. 2022. Linearly mapping from image to text space. *arXiv preprint arXiv:2209.15162*.

Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. The lambada dataset: Word prediction requiring a broad discourse context. *arXiv preprint arXiv:1606.06031*.

German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. 2019. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32.

Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. 2017. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 2001–2010.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106.

Dustin Schwenk, Apoorv Khandelwal, Christopher Clark, Kenneth Marino, and Roozbeh Mottaghi. 2022. A-okvqa: A benchmark for visual question answering using world knowledge. In *European Conference on Computer Vision*, pages 146–162. Springer.

Amanpreet Singh, Vivek Natarjan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. 2019. Towards vqa models that can read. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8317–8326.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Chenfei Wu, Shengming Yin, Weizhen Qi, Xiaodong Wang, Zecheng Tang, and Nan Duan. 2023. Visual chatgpt: Talking, drawing and editing with visual foundation models. *arXiv preprint arXiv:2303.04671*.

Zhengyuan Yang, Linjie Li, Jianfeng Wang, Kevin Lin, Ehsan Azarnasab, Faisal Ahmed, Zicheng Liu, Ce Liu, Michael Zeng, and Lijuan Wang. 2023. Mm-react: Prompting chatgpt for multimodal reasoning and action. *arXiv preprint arXiv:2303.11381*.

Friedemann Zenke, Ben Poole, and Surya Ganguli. 2017. Continual learning through synaptic intelligence. In *International Conference on Machine Learning*, pages 3987–3995. PMLR.

Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*.

Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. 2023a. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592*.

Wanrong Zhu, Jack Hessel, Anas Awadalla, Samir Yitzhak Gadre, Jesse Dodge, Alex Fang, Youngjae Yu, Ludwig Schmidt, William Yang Wang, and Yejin Choi. 2023b. Multimodal C4: An open, billion-scale corpus of images interleaved with text. *arXiv preprint arXiv:2304.06939*.

Appendix

We organize implementation details and additional supporting experimental findings as follows:

- Appendix A includes reproducibility and code release information.
- Appendix B provides details on compute resources (GPU) and training time.
- Appendix C describes the implementation details for the alignment and fine-tuning phases of LLaVA.
- Appendix D provides details on the datasets used in this paper.
- Appendix E summarizes findings of additional supporting experiments and ablation studies.

A Reproducibility and Code Release

All datasets and pre-trained models utilized in this work are widely recognized and publicly available. To facilitate replication of our experiments, we will create a GitHub repository with codes and a dedicated website. The code will be released under an MIT license. We will also release our trained MLLMs (weights and configurations). These resources will provide the exact sequence of examples used for each continual learning experiment. Additionally, the Appendix contains a comprehensive description of the hyperparameters and optimizer settings employed in each experiment.

B Hardware and Computational Budget

We trained a total of 9 MLLMs on 6 datasets, over 170 experiments on 4 A100 40GB GPU, with an aggregated compute time of $\sim$ 3400 (wall clock) hours ($\sim$ 141 days).

C Implementation Details

We provide details below of the experimental settings for the alignment and fine-tuning phases of the LLaVA1.5 training protocol we employ. Our configuration details are the same as LLaVA1.5 in order to reproduce and compare effectively to the LLaVA1.5 multi-modal training protocol.

C.1 Libraries and Tools

To reproduce the LLaVA 1.5 experiment, we build on top of the Prismatic library (Karamcheti et al., 2024), and use the Prismatic library for vision-language tasks and EleutherAI's LM_Eval library (Gao et al., 2023) for natural language evaluations. All our code is written in PyTorch (Paszke et al., 2019).

C.2 LLaVA Training Details

Following the LLaVA 1.5 protocol (Liu et al., 2023b), the visual encoder is a CLIP VIT-L@336px which takes an image resolution of 336px with letterbox resizing, and the alignment network is a two-hidden layer MLP projector with GELU activation. The LLM generations are limited to a maximum token length of 2048.

C.2.1 Alignment Stage

In the alignment stage, we use a learning rate of 0.001 with a linear warmup followed by a cosine decay scheduler. The training process is carried out for 1 epoch with a global batch size of 256, distributed as 16 samples per device. We use gradient checkpointing and mixed precision training to speed up computation. We use FSDP (Fully Sharded Data Parallel) (Zhao et al., 2023) to train all our models. During the alignment stage, we shard only the gradients and optimizer states and not the parameters.

C.2.2 Fine-tuning Stage

For the fine-tuning stage, the learning rate is $2e-05$, with a linear warmup for a 0.03 ratio of steps followed by cosine decay. This stage also runs for 1 epoch but with a smaller global batch size of 128, keeping the per device batch size constant at 16. Depending on the size of the LLM (Phi2 3B vs Pythia 160M), we vary the per device batch size, but the global batch size is kept constant across all experiments. We switch to the FSDP full sharding strategy, with all parameters, optimizer states, and gradients sharded across the devices.

C.3 Continual LLaVA

For the Continual LLaVA setting, we follow the same experimental configuration as before in the

LLaVA setting. A model trained on Task (i), will then be simply used as the pretrained checkpoint for Task $(i+1)$, with training following normally. The training configurations are then identical to the fine-tuning stage. In the case of PET methods like LoRA, the adapter weights are merged back into the LLM before starting training on the new task.

C.4 Hyperparameter Search

C.4.1 LLaVA1.5 Setting

To reproduce and directly compare against the LLaVA1.5 protocol, we keep the explicit training configurations the same (as mentioned in Appendix C).

C.4.2 LoRA

We train several different LoRA settings on the smallest Pythia-160M LLM for tractability and compare the resulting VL and NLU/NLG performances. We vary the a) target modules (between 1) all linear layers, and 2) key, query, and value projection layers only), b) LoRA ranks in the range of $1/4 - 1/2$ of the full rank of the model, rank stabilized LoRA (Kalajdzievski, 2023), and larger alpha values (16 instead of 8 as default). Table 6 shows the comparisons by training on the LLaVA setting.

C.4.3 Soft Targets

We train the LLaVA recipe with the soft targets with varying alpha $\alpha \in \{0.001, 0.01, 0.1\}$, and report results in Figure 7. The Pythia 160M model is used for this tuning.

D Dataset Preparation

D.1 Continual LLaVA

All datasets are primarily in the English language. However, some small samples of other languages such as Chinese can be found in the *ShareGPT* subset of LLaVA Instruct (Liu et al., 2023b), for instance. The preprocessing steps required for splitting the LLaVA1.5 Instruct json into component datasets, to build our continual learning setting, are detailed below: The primary LLaVA instruct json `llava_v1_5_mix665k.json` (Liu et al., 2023b) is split into component datasets based on the data entries as follows:

- **LLaVA Instruct (Liu et al., 2023b)**: The first 157,712 entries are directly assigned to the LLaVA subset, per manual verification.
- **ShareGPT (SG) (Liu et al., 2023a)**: Entries without an associated image are categorized under ShareGPT.
- **GQA (Hudson and Manning, 2019), OCR-VQA, TextVQA (Singh et al., 2019), Visual Genome (VG) (Krishna et al., 2017)**: Entries associated with images from the GQA, OCR-VQA, TextVQA, and `VG_100K` are mapped correspondingly to their respective datasets.
- **COCO-Based Datasets**: For images related to the COCO dataset, further categorization is performed based on the prompts:
  - **OKVQA** (Marino et al., 2019): If the prompt contains the phrase "Answer the question using a single word or phrase" and the image ID matches those in the OKVQA set.
  - **VQA-V2** (Goyal et al., 2017): If the above condition is not met, the entry is categorized under VQA-V2.
  - **A-OKVQA** (Schwenk et al., 2022): If the prompt contains multiple choice options ('A.', 'B.', 'C.').
  - **RefCOCO** (Kazemzadeh et al., 2014): If the prompt requests bounding box coordinates or a short description of a region.

The splitting process categorizes the dataset into distinct subsets as shown in Table 8, which are saved as separate JSON files in the specified directory. The number of entries in each subset is printed for verification.

After splitting, the datasets are then combined, based on the grouping in Table 5, to form the respective sequence of Tasks 1 to 5.

D.2 Evaluation Dataset Splits

We use the *slim* [3] versions of all VQAv2, GQA, Text-VQA and RefCOCO datasets for evaluation, as provided within Prismatic-VLMs (Karamcheti et al., 2024). All *slim* versions of the evaluation sets contain 1024 examples each, and we use the provided index splits for testing.

E Additional Experiments

E.1 Analysis of Forgetting across NLU and NLG tasks

To understand the composition of linguistic forgetting on the LLaVA setting, we look at the NLU and

[3] https://github.com/TRI-ML/vlm-evaluation

Table 6: **Analysis of LoRA Ranks and Configuration:** We train the Pythia 160M model with a varying set of ranks and configurations.

Model	Vision-Language (VL)				VL Avg.	NL Avg.	
	VQAv2	TextVQA OCR	TextVQA Pure	GQA	Acc ↑	Δ ↓	Acc ↑
Original LLaVA	30.32	2.40	3.83	22.17	5.29	7.83	24.78
Language Only LLM	0.00	0.00	0.00	0.00	0.00	-	32.61
LoRA (1/2 Full Rank, Higher Alpha)	28.72	1.05	2.67	19.73	2.84	9.33	23.28
LoRA (1/2 Full Rank, RSLoRA)	28.97	1.02	1.74	17.97	2.42	1.69	30.92
LoRA (1/4 Full Rank)	24.64	0.93	1.41	15.04	2.11	11.64	20.97
LoRA (1/4 Full Rank, Higher Alpha)	6.46	0.68	0.55	2.44	1.04	2.53	30.08
LoRA (1/2 Full Rank)	0.13	0.20	0.10	0.00	0.00	-	-
LoRA (1/2 Full Rank, RSLoRA, KQV Target)	0.00	0.00	0.00	0.00	0.00	-	-

Table 7: **Selecting α for Soft Targets**. We train the Pythia 160M model with Soft Targets by varying the $alpha \in \{0.001, 0.01, 0.1\}$.

Model	Vision-Language (VL)				VL Avg.	NL Avg.	
	VQAv2	TextVQA OCR	TextVQA Pure	GQA	Acc ↑	Δ ↓	Acc ↑
Language Only LLM	0.00	0.00	0.00	0.00	0.00	-	32.61
Soft Targets ($\alpha = 0.1$)	3.38	1.19	1.32	1.76	1.62	0.67	31.95
Soft Targets ($\alpha = 0.01$)	32.67	6.92	6.10	25.39	10.57	2.83	29.78
Soft Targets ($\alpha = 0.001$)	25.12	2.17	1.73	14.84	3.49	4.97	27.64
Original LLaVA	30.32	2.40	3.83	22.17	5.29	7.83	24.78

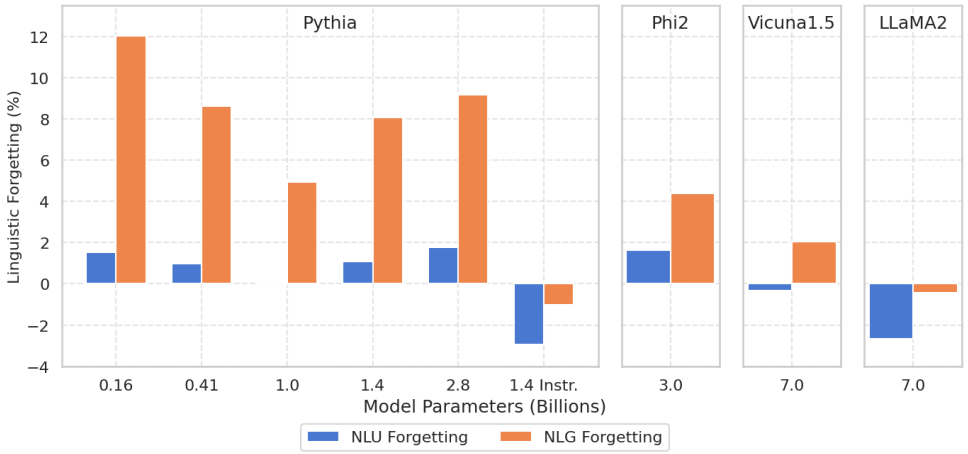


Figure 5: **Linguistic forgetting by NLU and NLG tasks**: For the LLaVA setting, we look at different model scales and families, and show the degree of linguistic forgetting by both NLU and NLG tasks separately. Here, the NLU and NLG averages are computed as the simple mean.

NLG forgetting for 9 different LLMs with varying scales: Pythia scaling family of models from the 160M to the 2.8B parameter range, Phi2 3B, Vicuna1.5 7B and LLaMA2 7B. Figure 5 shows these results. We can observe a clear trend of higher forgetting for the NLG dataset (Lambada), compared to forgetting for NLU. Another trend we note is that forgetting typically reduces with higher model scales, for both NLG and NLU.

E.2 Pre-trained LLMs

Here we a provide a list of the Pre-trained LLMs and their respective download links below: phi-2-3b: https://huggingface.co/microsoft/phi-2, pythia-160m: https://huggingface.co/EleutherAI/pythia-160m-deduped, pythia-410m: https://huggingface.co/EleutherAI/pythia-410m-deduped, pythia-1b: https://huggingface.co/EleutherAI/pythia-1b-deduped, pythia-

Table 8: **Dataset Split generated based on LLaVA1.5 Instruct composition (Liu et al., 2023a).**

Subset	Number of examples
LLaVA	157,712
SG	40,688
GQA	72,140
OKVQA	8,994
OCRVQA	80,000
A-OKVQA	66,160
TextCaps	21,953
RefCOCO	48,447
VG	86,417
VQA-V2	82,787

1p4b: https://huggingface.co/EleutherAI/pythia-1.4b-deduped, pythia-1p4b-instruct: https://huggingface.co/lambdalabs/pythia-1.4b-deduped-synthetic-instruct, pythia-2p8b: https://huggingface.co/EleutherAI/pythia-2.8b-deduped, llama2-7b-pure: https://huggingface.co/meta-llama/Llama-2-7b-hf, vicuna-v15-7b: https://huggingface.co/lmsys/vicuna-7b-v1.5.

E.3 Artifact Use Consistent With Intended Use

We ensure that any artifacts (such as datasets, software, models, code, or other supplementary materials) associated with our paper are used in a manner that aligns with their original purpose and the guidelines set forth by the creators. In particular, our artifacts are the models and code we build our experiments on top of, which we have listed above in Sections C.1 and E.2.